

Рубрика: Языки, компиляторы, системы программирования и аппаратно-зависимые вычисления
УДК 004.222

GoldenFloat: семейство форматов с плавающей точкой со
статическим разбиением,
выведенным из золотого сечения φ , от GF4 до GF256
с точным целочисленным тождеством Люка

Дмитрий Васильев
Независимый исследователь, Ко Самуи, Таиланд
Эл. почта: admin@t27.ai ORCID: 0009-0008-4294-6159

2026-06-02 v1.9

Поступила в редакцию: _____ После доработки: _____ Принята к
публикации: _____
DOI: _____ (присваивается редакцией и издательством)

Аннотация

Что сообщается в этой статье. Мы представляем аппаратно-ориентированное описание GoldenFloat (GF) — семейства форматов с плавающей точкой со статическим разбиением, порожденного единым замкнутым правилом, — и три конкретных артефакта: (i) открытый многоразрядный RTL-генератор, охватывающий GF4–GF256, с дифференциальной прогонкой в режиме непрерывной интеграции относительно корректно округлённого эталона; (ii) целочисленный тракт аккумулятора, точный по числам Люка, проверенный с точностью до 500 знаков для $n = 1, \dots, 256$; и (iii) кодек GF16 на FPGA, проходящий тестовое окружение 35 из 35 на частоте 323 МГц на Artix-7 (Xilinx XC7A100T). Оракул соответствия формата (Corona) поставляется в том же репозитории и используется как проверка чёрного ящика в нашем аудите непрерывной интеграции. Правило и область его применения. Для каждой полной разрядности $N \geq 4$ ширина поля порядка равна $e = \text{round}((N - 1)/\varphi^2)$ при ширине поля мантиссы $f = N - 1 - e$ и $\varphi = (1 + \sqrt{5})/2$. Правило воспроизводит реализованные ширины поля порядка девяти форматов GF4, GF8, GF12, GF16, GF20, GF24, GF32, GF64, GF256 (9/9) и согласованно распространяется на GF6, GF10, GF14, GF48, GF96, GF128, GF512, GF1024. Правило позиционируется рядом с posit (стандарт Posit 2022 года), takum (Hunhold 2024, 2025), OCP-MX (Rouhani и др. 2023) и черновиком многоразрядного формата с плавающей точкой IEEE P3109 — все они являются семействами, охватывающими ряд разрядностей под параметризованным правилом. Мы не делаем никаких заявлений о точности или превосходстве на отдельной ступени по сравнению с любым из них.

Что остаётся открытым. Тезис о широте/согласованности инструментальной цепочки зафиксирован как открытая гипотеза с предрегистрированным путём фальсификации: эксперимент на FPGA на согласованном субстрате и программная абляция при согласованном бюджете. Реестр фальсификации (FL-002) фиксирует открытые вопросы и эксперименты, которые их разрешили бы. Errata о корректности RTL от 2026-05-31 сообщается в разделе 5.5; изготовленные кристаллы TTSKY26b несут дефектный портфель умножителей, а исправленный генератор является базовой версией для регенерации.

Ключевые слова: числовые форматы; плавающая точка; золотое сечение; числа Люка; posit; takum; OCP-MX; арифметическое аппаратное обеспечение.

1 Введение

Числовые форматы для машинного обучения и научных вычислений диверсифицировались далеко за пределы IEEE 754. Доминирующая ось проектирования — это то, как фиксированный бюджет из N бит разбивается между динамическим диапазоном (поле порядка) и точностью (поле мантиссы) и как это разбиение масштабируется при изменении N . Три точки сравнения задают рамки настоящей заметки.

Posit. Gustafson и Yonemoto (2017) ввели posit с кодированием «режим плюс порядок», параметризованным размером поля порядка es . В литературе встречаются два различных расписания es : нестандартное расписание с $es = 0, 1, 2, 3, 4$ при разрядностях 8, 16, 32, 64, 128 (de Dinechin и др., 2019) и ратифицированный стандарт Posit 2022 года, который фиксирует $es = 2$ для всех разрядностей. Таким образом, динамический диапазон posit масштабируется вместе с полем режима, которое само является функцией представляемого значения.

Takum. Hunhold (2024, 2025) усовершенствовал идею конусного (tapered) формата при помощи единого вогнутого правила проекции, определённого единообразно для всех длин слова. Проекция takum отображает полный целочисленный диапазон N -битного слова на интервал числовой прямой; ширина поля мантиссы варьируется поэлементно, а динамический диапазон масштабируется с N в рамках одного аналитического отображения. Takum обратно совместим с IEEE-754 при $N \in \{16, 32, 64\}$ в том смысле, что оболочка динамического диапазона содержит соответствующий двоичный тип IEEE, и он несёт единственную известную нам прошедшую рецензирование многоразрядную аппаратную реализацию (ARITH 2025).

ОСР-MX. Rouhani и др. (2023) стандартизировали блочно-масштабируемые 4-, 6- и 8-битные типы микромасштабирования. Здесь поэлементная мантисса коротка, но общий масштаб уровня блока несёт динамический диапазон. Семейство микромасштабирования является развёрнутым промышленным эталоном для форматов малой разрядности.

Место GoldenFloat. GoldenFloat исследует намеренно иную точку в этом пространстве проектирования: статическое разбиение, фиксированное для каждой разрядности и не зависящее от хранимого значения. Поэтому динамический диапазон GFN определяется полностью шириной поля порядка $e(N)$ и выбором смещения, а не каким-либо зависящим от значения кодированием. Это обменивает поэлементную адаптивность posit и takum на свойство, которое труднее получить в значение-адаптивных семействах: замкнутое аналитическое правило, единообразно фиксирующее разбиение порядок/мантисса по всей лестнице разрядностей, плюс целочисленное тождество аккумулятора, зависящее только от выбора основания, а не от конкретной разрядности.

Историческая преемственность. Работа продолжает отечественную линию исследований оригинальных систем представления чисел в вычислительной технике, начатую троичной ЭВМ «Сетунь» под руководством Н. П. Брусенцова (Вычислительный центр МГУ, 1958), — первой в мире машиной на симметричном троичном коде $(-1, 0, +1)$, а в «Сетунь-70» (1970) — с введённым троичным форматом «трайт» и оптимизированным диапазоном мантиссы нормализованного числа. GoldenFloat переносит этот дух поиска неканонических числовых представлений в современный контекст форматов малой разрядности для машинного обучения. По доступным открытым источникам, GoldenFloat — один из первых именованных числовых форматов российского авторства, доведённых до уровня публикации и аппаратной реализации (FPGA).

Мотивация разбиения бюджета в точке $1/\varphi^2$ опирается на три факта, ни один из которых не является оригинальным для данной работы:

- Алгебраическое соотношение $\varphi^2 = \varphi + 1$ устраняет постоянный множитель в кодере на золотом сечении (Golden Ratio Encoder) Daubechies, Gunturk, Wang и Yilmaz (IEEE TIT, 2010): бета-кодер с основанием φ допускает безмножительную реализацию робастного АЦП только на сложениях. Это инженерная причина предпочесть основание φ внутри диапазона робастных оснований; это не утверждение о том, что φ является единственным допустимым основанием.
- То же алгебраическое соотношение даёт классическое тождество Люка $\varphi^{2n} + \varphi^{-2n} = L_{2n}$ (Lucas, 1878; Бине), которое позволяет накапливать φ -масштабированные частичные суммы через целочисленную рекуррентность Люка. Это арифметическая причина, по которой φ -основанная лестница допускает целочисленный тракт аккумулятора без аппаратного полутипа.
- Отношение $1/\varphi^2 \approx 0.382$ попадает внутрь эмпирического интервала отношений, который воспроизводит реализованные ширины поля порядка GF (раздел 2). Это отношение разделяют 82 других значения в данном интервале; выбор именно φ мотивирован двумя приведёнными выше алгебраическими фактами, а не одним лишь численным воспроизведением.

Вклад данной заметки состоит из трёх аппаратно-программных артефактов плюс замкнутого лестничного правила, связывающего их воедино:

- (C1) Открытый многоразрядный RTL-генератор. Единый параметризованный генератор на Verilog выпускает кодеки и умножители для всей лестницы $N \geq 4$ (GF4–GF256) из одного шаблона (E, M, BIAS) , с шириной регистра произведения $[2M + 1 : 0]$ и округлением полупозиции вверх (round-half-up). Исправленный генератор (раздел 5.5) является базовой версией для регенерации и встроен в дифференциальную прогонку непрерывной интеграции относительно корректно округлённого эталона; см. раздел 5.2.
- (C2) Целочисленный тракт аккумулятора, точный по числам Люка. Классическое тождество Люка $\varphi^{2n} + \varphi^{-2n} = L_{2n}$ (Lucas 1878; Бине), проверенное символически и с точностью до 500 десятичных знаков для $n = 1, \dots, 256$, позволяет нести φ -масштабированные частичные суммы в беззнаковом целочисленном хранилище без аппаратного полутипа. Мы не претендуем на это тождество как на своё; мы заявляем инженерное следствие: оно допускает целочисленный аккумулятор без `quire` в стиле `posit` или регистра Кулиша (Kulisch) (раздел 4).
- (C3) Оракул соответствия формата (Cofopa), поставляемый как встроенный в дерево чёрный ящик. Cofopa — это детерминированный эталон для каждого формата, который проверяет каждый кодек и умножитель, выпускаемый (C1), относительно замкнутой спецификации (C2). Он используется как шлюз CI в репозитории GoldenFloat и является единственной точкой, обнаружившей дефект портфеля умножителей от 2026-05-31 (раздел 5.3, раздел 5.5).

Лестничное правило $e = \text{round}((N - 1)/\varphi^2)$, $f = N - 1 - e$ ($N \geq 4$) точно воспроизводит девять реализованных ширин поля порядка GF (9/9) и согласованно распространяется на GF6, GF10, GF14, GF48, GF96, GF128, GF512, GF1024. Битстрим GF16 на FPGA проходит тестовое окружение кодера 35 из 35 на частоте 323 МГц на Artix-7 (Xilinx XC7A100T); ни один физический кристалл не вернулся ни для одной ступени.

Мы рассматриваем это как исследовательскую заметку, а не как программный документ. Мы не утверждаем, что какая-либо ступень GF точнее равнонастроенного не- φ формата той

же разрядности; не утверждаем, что φ — единственное или привилегированное основание; и не заявляем о кремниевой валидации на какой-либо ступени. Там, где свидетельства неполны, мы говорим об этом, помечаем как открытое и приводим эксперимент, который это фальсифицировал бы. Раздел 2 определяет лестничное правило и сообщает об исчерпывающем поиске с поправкой на множественность (look-elsewhere). Раздел 3 фиксирует якорь кодера на золотом сечении и точно указывает, что он устанавливает, а что нет. Раздел 4 проверяет тождество Люка и мотивирует целочисленный аккумулятор. Раздел 5 сообщает о статусе аппаратной части по лестнице GF4–GF256. Раздел 6 соотносит GF с предшествующими работами. Раздел 7 даёт предрегистрацию открытых гипотез. Раздел 8 даёт реестр фальсификации FL-002. Приложения содержат верификационный скрипт, индекс форматов, таблицу поправки на множественность, предрегистрацию FPGA-эксперимента на согласованном субстрате и протокол репликатора.

2 Лестничное правило $e = \text{round}((N - 1)/\varphi^2)$

2.1 Определение

Пусть $N \geq 4$ — полная разрядность формата в битах, включая один знаковый бит. Разбиение GoldenFloat выделяет

$$e = \text{round}\left(\frac{N - 1}{\varphi^2}\right), \quad f = N - 1 - e, \quad (1)$$

где $\varphi = (1 + \sqrt{5})/2 = 1.6180339887\dots$ и $\varphi^2 = \varphi + 1 = 2.6180339887\dots$, так что $1/\varphi^2 = 0.3819660\dots$. Разбиение статично: e зависит только от N и никогда от представляемого значения. В таблице 1 приведён результат для семнадцати форматов (девять реализованных разрядностей плюс восемь разрядностей-расширений). При $N \rightarrow \infty \text{round}((N - 1)/\varphi^2)/(N - 1) \rightarrow 1/\varphi^2$, и реализованное отношение в таблице 1 сходится соответственно. Для $N \geq 4$ нет граничной неустойчивости.

Краевые случаи при $N = 2, 3$. Применение правила при $N = 2$ даёт $e = 0$ — знаковое однобитное поле без порядка, что не является представлением с плавающей точкой в обычном смысле. При $N = 3$ оно даёт $e = 1$, $f = 1$ — четырёхзначный формат с пренебрежимо малым диапазоном и точностью. Поэтому мы определяем лестницу как от GF4 до GF256 и рассматриваем GF2/GF3 как вырожденные краевые случаи формулы, а не как реализованные форматы. Все утверждения настоящей заметки касаются $N \geq 4$.

2.2 Поправка на множественность (look-elsewhere) для совпадения с φ^{-2}

Сначала отрицательный результат. $\varphi^{-2} \approx 0.38197$ невозможно отличить от прочих 82 совпадающих отношений на основании представленных здесь численных свидетельств. Приведённая ниже поправка на множественность (look-elsewhere) делает это точным.

Постановка. Мы искали отношения $r \in [0.1, 0.9]$ с шагом 10^{-5} , что даёт мощность пространства поиска $N_s = 80\,000$. Каждое отношение проверялось против $W = 9$ реализованных разрядностей GoldenFloat; отношение совпадает, если оно воспроизводит все 9 разрядностей. Поиск вернул $K = 83$ совпадающих отношения.

Семейственная вероятность. При нулевой гипотезе о том, что каждое отношение проходит тест каждой разрядности независимо с равномерной поразрядной вероятностью p , откалиброванной так, что $\mathbb{E}[\text{совпадений}] = K$, имеем $p_{\text{match}} = K/N_s = 83/80\,000 \approx 1.04 \times 10^{-3}$.

Таблица 1: Лестничное правило GF для семнадцати форматов. Столбцы: полная разрядность N , биты порядка e , биты мантиссы $f = N - 1 - e$, сырое значение $(N - 1)/\varphi^2$ до округления и реализованное отношение $e/(N - 1)$ (цель $1/\varphi^2 = 0.38197$). Верхний блок перечисляет девять разрядностей с вернувшимся кремнием или финализированным RTL. Средний блок (GF6, GF10, GF14, GF48, GF96, GF128) перечисляет выведенные из правила ступени без вернувшегося кремния; RTL для GF128 написан, но не отправлен на изготовление. Нижний блок (GF512, GF1024) сообщает о двух ступенях-расширениях правила, чьи смещения ($2^{194} - 1$, $2^{390} - 1$) превышают поле u128, используемое в записи ROM Corona, и отслеживаются символически только на уровне оракула t27 SSOT. Все семнадцать строк арифметически проверены по уравнению (1). Значения вычислены с точностью 200 знаков в mpmath.

N	e	f	$(N - 1)/\varphi^2$	$e/(N - 1)$
4	1	2	1.1459	0.33333
8	3	4	2.6738	0.42857
12	4	7	4.2016	0.36364
16	6	9	5.7295	0.40000
20	7	12	7.2574	0.36842
24	9	14	8.7852	0.39130
32	12	19	11.8409	0.38710
64	24	39	24.0639	0.38095
256	97	158	97.4013	0.38039
6	2	3	1.9098	0.40000
10	3	6	3.4377	0.33333
14	5	8	4.9656	0.38462
48	18	29	17.9524	0.38298
96	36	59	36.2868	0.37895
128	49	78	48.5097	0.38583
512	195	316	195.1846	0.38160
1024	391	632	390.7512	0.38221

Число совпадений $X \sim \text{Binomial}(N_s, p_{\text{match}})$. Вычисляя через регуляризованную неполную бета-функцию с точностью 60 знаков:

$$P(X \geq 83) \approx 7.1 \times 10^{-3}.$$

Таким образом, семейственная вероятность наблюдения по меньшей мере 83 совпадений при данной нулевой гипотезе не пренебрежимо мала; совпадающее множество является умеренно частым исходом поиска, а не ярким событием в хвосте распределения.

Поправка Бонферрони для φ^{-2} . Нескорректированная вероятность того, что любое заранее заданное отношение попадёт в совпадающее множество, равна $p_{\text{match}} \approx 1.04 \times 10^{-3}$. После поправки Бонферрони для $N_s = 80\,000$ одновременных тестов скорректированное р-значение насыщается на единице (поскольку $N_s \cdot p_{\text{match}} = K = 83$). φ^{-2} не значимо по Бонферрони.

Сужение с 12 форматами. Расширение поиска на все двенадцать форматов в таблице 1 сужает множество кандидатов с 392 отношений (девятиформатный интервал $[0.37844, 0.38235]$) до 47 отношений (двенадцатиформатный интервал $[0.38189, 0.38235]$) — сокращение в 8.3×. Единственность при более тонком разрешении требует точной интервальной арифметики и здесь не заявляется.

Вывод. Численное совпадение того, что φ^{-2} удовлетворяет лестничному правилу, реально, но не доказательно: 82 других отношения удовлетворяют тому же правилу. Аргументация в пользу φ опирается на его структурные свойства (алгебраическое самоподобие, рекуррентность Фибоначчи, точное по Люка накопление), а не на одно лишь совпадение в пространстве поиска. Все вычисления воспроизводимы через `look_elsewhere_calc.py` в дополнительных материалах.

2.3 Режим округления

Правило использует округление к ближайшему чётному, но этот выбор несущественен для каждой реализованной разрядности, как фиксирует сноска.¹

3 Безмножительный якорь GRE

Независимая, прошедшая рецензирование мотивация для φ исходит из кодера на золотом сечении (Golden Ratio Encoder, GRE) Daubechies, Gunturk, Wang и Yilmaz, “The Golden Ratio Encoder”, IEEE Transactions on Information Theory 56(10):5097–5110, 2010 (arXiv:0809.1257). GRE — это схема аналого-цифрового преобразования, а не формат с плавающей точкой. То, что он устанавливает и что мы заимствуем, — это точное инженерное свойство основания φ :

φ — это основание, при котором бета-кодер допускает безмножительную реализацию робастного АЦП только на сложениях, поскольку $\varphi^2 = \varphi + 1$ устраняет компоненту с постоянным множителем (Daubechies и др., IEEE TIT 2010).

3.1 Что результат GRE устанавливает, а что нет

Мы явно очерчиваем область действия, поскольку два естественных перетолкования ложны.

Нет утверждения, что выбор основания вынужден. Мы не утверждаем, что φ — единственное допустимое основание, дающее работоспособный кодер. В той же статье анализируются другие значения β и демонстрируется диапазон устойчивых и робастных оснований. Любое утверждение о том, что φ является единственным допустимым основанием, было бы ложным и здесь не делается.

Нет точечной робастности при $\beta = \varphi$. Чистый алгоритмический кодер при $\beta = \varphi$ устойчив, но не робастен. Робастность установлена только для реализации с диапазоном параметров / ориентированным ациклическим графом (Daubechies и др., теорема 8), а не для единственной точки $\beta = \varphi$. Мы не смешиваем это с анализом восстановления β у Ward (2008, arXiv:0806.1083), который является другой статьёй, обращающейся к другому вопросу, и не является источником безмножительного свойства, используемого здесь.

Поэтому связь, которую мы проводим, узка и защитима: $\varphi^2 = \varphi + 1$ — это алгебраический факт, устраняющий постоянный множитель в рекурсии GRE, и тот же алгебраический факт лежит в основе тождества Люка из раздела 4. Это инженерная причина предпочесть φ^2 внутри совпадающего интервала из раздела 2.2, а не утверждение о том, что кодер однозначно выбирает φ .

¹Для всех девяти реализованных разрядностей сырое значение $(N-1)/\varphi^2$ не является полуцелым (ближайшее приближение — $N = 64$ при 24.064 и $N = 128$ при 48.510), поэтому выбор режима округления не влияет ни на одну реализованную разрядность. Тем не менее мы задаём округление к ближайшему чётному (по умолчанию в IEEE 754) для формальной полноты.

4 Точное целочисленное тождество Люка

4.1 Мотивация: большие накопления и целочисленные суммы

Практическая забота для числовых форматов малой разрядности — поведение потери значимости при больших накопленных суммах. В арифметическом тракте с фиксированной точностью с плавающей точкой и шириной мантииссы f относительная погрешность длинного тракта умножения с накоплением растёт по мере увеличения числа накопленных слагаемых: каждое сложение округляет текущую сумму до $f + 1$ значащих бит, а абсолютная погрешность масштабируется (в худшем случае) с величиной частичной суммы. Это хорошо понятая мотивация за конструкциями точного накопления, такими как `quire` в `posit` (Gustafson и Yonemoto, 2017) и аккумулятор Кулиша (Kulisch), оба из которых дополняют плавающий тракт внешним широким регистром с фиксированной точкой, чтобы длинные суммы могли вычисляться без промежуточного округления.

Арифметическое свойство, которое мы используем в разделе 4.2 ниже, даёт иной путь к той же цели, специфичный для основания φ : φ -масштабированные частичные суммы вида $\sum_i \varphi^{2n_i}$ можно отслеживать через целочисленную рекуррентность по числам Люка, причём сопряжённый остаток $\sum_i \varphi^{-2n_i}$ точен и ограничен. Аппаратное следствие в том, что длинный аккумулятор, построенный вокруг основания φ , не требует внешнего широкого регистра с фиксированной точкой или аппаратного полутипа; целочисленное хранилище и рекуррентность Люка сами являются аккумулятором.

Мы излагаем это здесь как инженерную причину интереса к тождеству до доказательства и возвращаемся в разделе 4.4 к тому, что заявляется и что не заявляется о конкретном аппаратном аккумуляторе.

4.2 Формулировка и классическое происхождение

Предложение 1 (Точное по Люка тождество для φ^2). Для каждого целого $n \geq 1$,

$$\varphi^{2n} + \varphi^{-2n} = L_{2n}, \quad (2)$$

где L_k — k -е число Люка ($L_0 = 2$, $L_1 = 1$, $L_k = L_{k-1} + L_{k-2}$).

Это классическое следствие формулы Бине $L_k = \varphi^k + (-\varphi)^{-k}$: для чётного $k = 2n$ знаковый множитель $(-1)^k$ обращается, оставляя ровно целое L_{2n} . Случай $n = 1$ даёт якорь

$$\varphi^2 + \varphi^{-2} = 3 = L_2, \quad (3)$$

который мы приписываем Lucas (1878) и формуле Бине и никогда не представляем как оригинальный для данной работы или её автора.

4.3 Верификация (F1)

Мы проверили уравнение (2) двумя независимыми способами для $n = 1, \dots, 256$ (так что $2n = 2, \dots, 512$):

1. Символически. Используя точную арифметику в $\mathbb{Q}[\sqrt{5}]$ (SymPy), алгебраическое тождество $\varphi^{2n} + \varphi^{-2n} - L_{2n} = 0$ выполняется для каждого n в диапазоне.
2. Численно. При 500 десятичных знаках (mpmath) максимальный остаток $|\varphi^{2n} + \varphi^{-2n} - L_{2n}|$ по диапазону равен 1.55×10^{-499} при $n = 256$, значительно ниже целочисленного масштаба. Это уровень численного шума, согласующийся с точностью 500 знаков.

Все 256 точных проверок SymPy проходят тождественно; наихудший относительный остаток равен 1.55×10^{-499} при $n = 256$. Сокращённая таблица результатов (избранные строки от $n = 1$ до $n = 256$) приведена в приложении А. Скрипт воспроизведён в приложении А.

4.4 Инженерное следствие и что не заявляется

Предложение 1 означает, что степени φ^2 удовлетворяют линейной целочисленной рекуррентности. Следовательно, φ -масштабированные частичные суммы вида $\sum_i \varphi^{2n_i}$ в принципе можно накапливать в беззнаковом целочисленном хранилище, отслеживая соответствующие целые числа Люка L_{2n_i} , при этом остаточные члены φ^{-2n_i} отслеживаются символически или ограничиваются. Это арифметическая основа целочисленного тракта накопления, не зависящего от аппаратного полутипа и не требующего внешнего широкого регистра с фиксированной точкой, такого как `quire` или аккумулятор Кулиша.

Мы ясно заявляем, что не утверждается. Конкретный аппаратный аккумулятор, реализующий эту идею, с заданными длинами слов, границами переполнения, задержкой и сравнением с `quire posit` или аккумулятором Кулиша, является будущей работой и здесь не заявляется. Само тождество проверено (раздел 4.3); инженерный артефакт — нет. Relevantная предшествующая работа по целочисленной арифметике в базе Фибоначчи/ φ — это Ahlbach, Usatine и Pippenger, “Efficient Algorithms for Zeckendorf Arithmetic” (2012, arXiv:1207.4497), которую мы цитируем как якорь алгоритмической осуществимости целочисленной арифметики в базе φ .

5 Статус аппаратной части по лестнице

5.1 Определение лестницы готовности

Мы сообщаем о статусе аппаратной части относительно шестиступенчатой лестницы готовности, взятой дословно из документа STATUS репозитория tt-trinity-phi:

1. SPEC — существует версионированная числовая/поведенческая спецификация.
2. RTL — синтезируемый Verilog, реализующий спецификацию, зафиксирован в репозитории.
3. SIM — функциональная симуляция (`cocotb + iverilog`) проходит канонический якорь.
4. SYNTH — синтез Yosys + чистый линт Verilator.
5. GDS/TAPEOUT — поток OpenLane2 (SKY130A) производит артефакт `tt_submission`; отправка на shuttle принята.
6. SILICON — физический кристалл измерен и охарактеризован на плате.

Более высокая ступень заявляется только если все нижние ступени удовлетворены с доказательствами внутри публичного репозитория.

5.2 Статус по разрядностям

Таблица 2 сообщает о статусе по разрядностям, проверенном прямой инспекцией ветки `master gHashTag/t27` и ветки `main gHashTag/tt-trinity-phi 2026-05-31`. В таблице “TTSKY26b” обозначает принятие в отправку на shuttle TTSKY26b (SKY130A); счётчики байт для каждого RTL-файла опущены ради вёрстки и зафиксированы в репозитории.

Разрядность GF16 — единственная ступень с документированным битстримом FPGA и функциональной симуляцией, проходящей канонический якорь скалярного произведения `0x47C0`. Все десять разрядностей от GF4 до GF256 несут синтезируемый RTL на Verilog-2005 в `gHashTag/tt-trinity-phi/src/` и были включены в отправку на shuttle TTSKY26b в Tiny Tapeout (SKY130A, отправлено 2026-05-17). Shuttle TTSKY26a (ядро сетки `dot4 GF16`) был отправлен ранее с изготовлением, запланированным на 2026-10-28. Ни один физический кристалл не был возвращён, охарактеризован или измерен ни для одной ступени; поэтому любая цифра «на ватт», «на джоуль» или пиковой пропускной способности вне

Таблица 2: Статус аппаратной части по разрядностям по лестнице GF, проверенный инспекцией репозитория (gHashTag/t27 master и gHashTag/tt-trinity-phi main, 2026-05-31). Строки, помеченные †, имели дефект умножителя на уровне генератора в RTL, отправленном в TTSKY26b 2026-05-17; дефект раскрыт в разделе 5.5, а исправленный RTL опубликован в gHashTag/tt-trinity-gamma PR#110 и gHashTag/t27 PR#1024.

Разрядность	RTL-файл(ы)	SIM	SYNTH	GDS shuttle	Кремний
GF4	gf4_add.v	только RTL	только RTL	TTSKY26b	не вернулся
GF8	gf8_add.v†	только RTL	только RTL	TTSKY26b	не вернулся
GF12	gf12_add.v†	только RTL	только RTL	TTSKY26b	не вернулся
GF16	gf16_add.v, gf16_dot4.v, gf16_mul.v†; битстрим FPGA, Artix-7, 35/35 на 323 МГц	да (0x47C0)	да	TTSKY26b*	не вернулся
GF20	gf20_add.v†	только RTL	только RTL	TTSKY26b	не вернулся
GF24	gf24_add.v†	только RTL	только RTL	TTSKY26b	не вернулся
GF32	gf32_add.v†	только RTL	только RTL	TTSKY26b	не вернулся
GF64	gf64_add.v†	только RTL	только RTL	TTSKY26b	не вернулся
GF128	gf128_add.v†	только RTL	только RTL	TTSKY26b	не вернулся
GF256	gf256_add.v†	только RTL	только RTL	TTSKY26b	не вернулся

* GF16 был также отправлен ранее в shuttle TTSKY26a.

архитектурных проекций является спроектированной, а не измеренной. DOI аппаратного архива 10.5281/zenodo.19227877 цитируется только как архив RTL и артефактов отправки на shuttle, но никогда как ссылка на производительность или результаты.

5.3 Оракул соответствия формата (Corona)

Сопутствующий кристалл только для чтения, gHashTag/tt-trinity-corona, готовится для shuttle TTGF26a Tiny Tapeout (GF180MCU 180 нм, 4×4 тайла, целевая отправка 2026-06-22, ожидаемый кремний 2026-10 – 2026-11). Corona — четвёртый кристалл в линейке TRI-NET после Phi, Euler и Gamma. Это оракул соответствия формата, а не вычислительный ускоритель: запрос приходит как 7-битный индекс формата на `in[6:0]`, и чип возвращает запрошенные поля записи внутрикристального ROM. ROM кодирует снимок из 80 записей числовых форматов, отобранный из единого источника истины TRI-NET (gHashTag/t27 specs/numeric/formats_catalog.t27); это подмножество каталога, а не его полный объём (каталог SSOT на момент данной ревизии содержит 83 формата — см. сопутствующую статью о каталоге). 80 записей ROM обслуживаются 17 уникальными модулями RTL-декодеров Tier-1, покрывающими 18 семейств форматов (некоторые семейства разделяют декодер, например FP8 E4M3 переиспользует декодер MXFP8 E4M3), каждый из которых преобразует внутрикристальный формат в IEEE 754 FP32 или INT32. 80 записей разбиты на тринадцать кластеров форматов (двоичный IEEE, десятичный IEEE, ML низкой точности, GoldenFloat, posit/unum-III, OCP-MX, LNS, целый/фиксированный, исторический, теоретический, сжатие, расширенный, квантово-настроенный). Текущая конструкция синтезируется приблизительно в 2,308 ячеек ($\approx 1\%$ бюджета сайтов 4×4). HEAD главной ветки репозитория на момент данной ревизии — b6944b29daca.

Мы заявляем, чем Corona не является, чтобы сохранить честность настоящей заметки. Corona не делает никаких заявлений о вычислительной пропускной способности, энергии на операцию или производительности относительно любого другого чипа-ускорителя, и её существование не является свидетельством того, что φ -лестница или любой формат GoldenFloat превосходит конкурирующую числовую систему. Тезис о широте-как-рве (breadth-as-moat) FL-002 из раздела 8 остаётся [Открытая гипотеза], и Corona не меняет его статус. Takum (Hunhold 2024, arXiv:2412.20273) остаётся действующим живым контрпримером к FL-002 и поставляется в ROM Corona как запись Tier-2 (или Tier-1, если лицензиро-

вание VHDL эталонной реализации Hunhold разрешится благоприятно); он не подавляется. Чип по построению только для чтения, поэтому никакой класс дефекта округления вида, раскрытого в разделе 5.5, не может применяться к его основному тракту чтения из ROM; 17 декодеров Tier-1 подвергаются тому же дифференциальному прогону RTL-аудита, что и остальной портфель, с пятью слоями доказательства на декодер (исчерпывающая прогонка, независимый эталон, формальная эквивалентность, постсиликоновый вектор, kill-мутации), покрывающими в общей сложности 592,308 входных кодов. На момент настоящей рукописи (51 направленный тест PASS, GDS и предпроверка Tiny Tapeout PASS, 21 CI-шлюз автообнаружен) готовность Corona находится на ступени GDS/TAPEOUT лестницы раздела 5 в ожидании окна отправки TTGF26a; ни один кремний Corona не был измерен.

5.4 Конфаундер аппаратного субстрата для программных сравнений BPB

Сравнения бит-на-байт (bits-per-byte, BPB) между GF16 и IEEE fp16, сообщаемые конвейером обучения IGLA RACE (gHashTag/trios-trainer-igla), производятся на субстратах — потребительских GPU и трактах CPU SIMD, — чьё аппаратное умножение с накоплением спроектировано и настроено под IEEE fp16 / bf16, с нативными TensorCore или AMX, диспетчеризующими fp16/bf16 за один такт. GF16 на том же субстрате эмулируется через целочисленно-кодированные тракты без нативного умножителя. Поэтому сообщаемый разрыв BPB порядка 10^{-1} между GF16 и fp16 на этом субстрате не измеряет формат GF16 — он измеряет формат GF16 на аппаратуре, оптимизированной против него. Справедливое сравнение BPB требует либо fp16-нативного кремния, спаренного с GF16-нативным кремнием при согласованном технологическом узле и площади, либо контролируемой FPGA-оснастки, где оба формата получают эквивалентный бюджет LUT/DSP. Пока такое спаренное измерение не существует, программно-субстратные дельты BPB в любом направлении фиксируются как искажённые субстратом и не являются свидетельством за или против широты-как-рва. Этот открытый вопрос зафиксирован как пункт (a) FL-002 в разделе 8 и является основной причиной, по которой ни одно заявление о точности на отдельной ступени нигде в данной заметке не появляется.

5.5 Errata о корректности RTL

2026-05-31 дифференциальная прогонка на уровне RTL относительно корректно округлённого эталона с плавающей точкой обнаружила систематический дефект на уровне генератора в большинстве умножителей GoldenFloat в виде, отправленном на shuttle TTSKY26b Tiny Tapeout 2026-05-17 (полная помодульная разбивка, вывод корневой причины и обсуждение объёма исправления приведены в приложении F). Регистр произведения в отправленном RTL умножителя был объявлен на два бита уже, поэтому для базового входа 1.0×1.0 старший бит был усечён, и результат читался как 0.5, реплицированный по всему сгенерированному портфелю. Исправленный универсальный генератор умножителя с шириной произведения $[2M + 1 : 0]$ проходит исчерпывающие прогонки на gf8 (26,360/26,360), gf12 (109,576/109,576), gf16 (262,144/262,144), gf20 (100,000/100,000), gf24 (100,000/100,000), gf32 (200,000/200,000) и направленные точные тесты на gf64/gf128/gf256 и теперь встроен в CI-шлюз (run_gf_audit.sh) при каждом коммите. Кремний TTSKY26b уже находится в изготовлении и не может быть отозван, поэтому кристаллы gamma и phi будут нести дефектный RTL умножителя; исправленный портфель является базовой версией для регенерации для следующего shuttle, и ни одно заявление о точности на отдельной ступени в данной заметке не зависело от умножителей в виде, в каком они были отправлены (раздел 5.2). Дефект не отменяет лестничную арифметику раздела 2, тождество Люка раздела 4 или учёт множественности раздела 2, все из которых являются абстрактными спецификациями, не зависящими от какой-либо конкретной реализации RTL.

5.6 Эмпирический вердикт на реальном корпусе

Программное прямое сопоставление между конвейером смешанной точности на φ -лестнице и гетерогенным «зоопарком» числовых форматов было выполнено на корпусе реального текста (`tiny_shakespeare` — наименьший публично аудируемый корпус, который мы смогли зафиксировать) внутри оснастки IGLA RACE 2026-05-31. Сначала и прямо фиксируем агрегированный вердикт: сравнение даёт недостаточно свидетельств. Данные не подтверждают заявление о превосходстве на отдельной ступени ни в одном из направлений.

Агрегированный результат (бит-на-байт; меньше — лучше). На `tiny_shakespeare` среднее ВРВ на отложенной выборке составило $\text{ВРВ}_\varphi = 5.9871$ (выборочное стандартное отклонение $\sigma = 0.0911$) для плеча φ -лестницы и $\text{ВРВ}_{\text{zoo}} = 6.0454$ ($\sigma = 0.2083$) для плеча гетерогенного зоопарка по $n < 11$ парным сидам. Двухрешенческий вердиктный комплект был таков: (i) сравнение фронта Парето при согласованном бюджете бит на вес возвращает зоопарк побеждает (точка зоопарка $P_w = 8.0$ достигает $\text{ВРВ} = 5.361$, $\text{CI} = [5.348, 5.375]$, тогда как точка φ -лестницы $P_w = 4.0$ достигает $\text{ВРВ} = 5.360$, $\text{CI} = [5.347, 5.373]$, причём два доверительных интервала перекрываются); (ii) нормализованный байесовский вторичный критерий возвращает ничью; (iii) третичная апостериорная достоверность возвращает $P(\varphi < \text{zoo}) = 0.976$, ниже предрегистрированного порога решения для заявления о точности на отдельной ступени. Поэтому агрегированный вердикт разрешается в недостаточно свидетельств. Мы не утверждаем, что φ -лестница точнее зоопарка при согласованном бюджете бит; мы также не утверждаем обратного.

Открытые пробелы, от которых зависит вердикт. Три пробела не позволяют повысить вердикт с «недостаточно свидетельств» до проверенного результата в любую сторону, каждый зафиксирован в реестре фальсификации (раздел 8, подпункт FL-002): (1) плечо зоопарка не диспетчеризует по P_w ; (2) нормализованная байесовская достоверность вычисляется, но не встроена в агрегат; и (3) регрессионная фиксация (`regression pin`) отсутствует. Размер выборки в этом прогоне ($n < 11$ парных сидов) ниже целевого минимально обнаружимого эффекта $n = 11$, требуемого для $\text{MDE} = 0.05$ ВРВ при $\alpha = 0.05$, мощность = 0.80.

Дифференциальный оракул точности округления (`host`, `unsealed`). Отдельно от корпусного ВРВ-эксперимента выше мы сообщаем покоэффициентный дифференциальный оракул точности округления: прямой `round-trip` $\mathbb{R} \rightarrow F \rightarrow \mathbb{R}$ через эталонный кодек GF16 (`conformance/gf16_ref.py`, E6M9) и через `ml_dtypes.bfloat16` и `numpy.float16` на эталонных распределениях протокола (сид 2718281, 2,000,000 выборок на распределение), с метрикой $\text{NMSE}(F) = \mathbb{E}[(x - Q_F(x))^2] / \mathbb{E}[x^2]$. Измеренные отношения согласованны по распределениям: GF16 имеет примерно в 16 раз меньший NMSE, чем `bfloat16` (отношение ≈ 0.063 ; 9 против 7 бит мантииссы), и примерно в 4 раза больший NMSE, чем `fp16` (отношение ≈ 4.0 ; 9 против 10 бит мантииссы) вблизи 1.0. На широком диапазоне ($\log_2 |x| \sim U(-28, 28)$) `fp16` переполняется на 21.4 % выборок (максимум $\approx 6.55 \cdot 10^4$), тогда как GF16 (максимум $\approx 4.29 \cdot 10^9$) и `bf16` не теряют ни одной — честный компромисс порядка / мантииссы. Эти числа измерены на хосте и не запечатаны (`unsealed`): это различие в бюджете бит мантииссы/порядка, а не заявление о кремнии или о превосходстве на отдельной ступени; конфаундер аппаратного субстрата (раздел 5.4) и пункт FL-002 (а) остаются в силе. Прямой результат воспроизводим (скрипт `repro/numerics/nmse_gf16.py`, манифест `nmse_manifest_protocol_v1.json`, соответствующий схеме `schemas/nmse-protocol-v1.json` в репозитории `gHashTag/t27`). Поле `seal_hash` манифеста честно установлено в `unsealed`, поскольку прогон выполнен на незапечатанном хосте. Результат согласуется с мотивацией E6M9 у IBM DLFLOAT (раздел 6.1) и Popescu и др.

6 Связь с предшествующими работами

GoldenFloat находится среди нескольких опубликованных усилий охватить диапазон разрядностей единым параметризованным правилом. Мы рассматриваем их как предшественников и союзников, а не как конкурентов, которых следует отвергнуть, и не утверждаем, что GF лучше любого из них по какой-либо оси.

Posit. Gustafson и Yonemoto (2017) ввели posit с кодированием «режим плюс порядок», параметризованным размером поля порядка es . В литературе встречаются два различных расписания es : нестандартное расписание с $es = 0, 1, 2, 3, 4$ при разрядностях 8, 16, 32, 64, 128 (de Dinechin и др., 2019) и ратифицированный стандарт Posit 2022 года, который фиксирует $es = 2$ для всех разрядностей. Аппаратные реализации включают PERCIVAL (Mallaseñ и др., 2022) и Big-PERCIVAL (Mallaseñ и др., 2024). Лестница posit является естественным контролем для вопроса широты (раздел 8).

6.1 GoldenFloat против Takum: структурированное сравнение бок о бок

Признание приоритета. Арифметика takum (Hunhold 2024, 2025) — ближайшая опубликованная альтернатива лестнице GoldenFloat с прошедшей рецензирование многоразрядной аппаратной реализацией. Статья Hunhold на ARITH 2025 представляет RTL- и FPGA-результаты для нескольких разрядностей takum под единым замкнутым правилом проекции, опережая любое сопоставимое аппаратное заявление для GoldenFloat. Лестница GoldenFloat в настоящее время не имеет бенчмарка точности на отдельной ступени против takum, не имеет прошедшей рецензирование аппаратной публикации и не имеет прямого численного сравнения. Таблица 3 фиксирует, что делает каждый формат и где стоит каждое заявление; это не конкурентный рейтинг.

В настоящее время: takum имеет прошедшие рецензирование аппаратные результаты на FPGA на нескольких разрядностях; GoldenFloat — нет. GoldenFloat заявляет свойство целочисленного аккумулятора, точного по Люка; takum это не нацелено. Ни один из форматов не имеет прямого бенчмарка численной точности против другого.

IEEE P3109. Рабочая группа IEEE P3109 с 2023 года разрабатывает черновик стандарта многоразрядного двоичного формата с плавающей точкой, нацеленного на ускорители машинного обучения. Текущий публичный рабочий черновик (v0.9.1, 2025) специфицирует двоичные форматы с плавающей точкой на тринадцати разрядностях (от 3 до 15 бит) под единым параметризованным правилом, с эталонной реализацией, поддерживаемой Graphcore (graphcore-research/gfloat), и формальной верификацией, сообщённой на ARITH 2025. P3109 — ближайшее действующее, отраслевое семейство форматов с плавающей точкой, охватывающее ряд разрядностей, к лестнице GoldenFloat и является прямой целью фальсификации для любого прочтения настоящей работы в духе широты-как-рва (раздел 8, FL-002 (a)).

IEEE DFloat и независимый формат 1/6/9 (E6M9). Побитовая раскладка GF16 — знак, 6-битный порядок, 9-битная мантисса (E6M9, BIAS = 31) — не является новой как раскладка и не заявляется таковой. IBM DFloat (Agrawal и др., ARITH 2019, DOI 10.1109/ARITH.2019.00023) — 16-битный формат с плавающей точкой ровно с 6 битами порядка и 9 битами мантиссы, предложенный для обучения и инференса глубоких сетей; это в точности раскладка полей GF16. Popescu, Sentieys и др. (arXiv:2103.15940) независимо выделяют разбиение 1/6/9 как точку компромисса между диапазоном и точностью для 16-битного обучения. Таким образом, выбор раскладки E6M9 имеет независимый прецедент, и GoldenFloat честно позиционируется как один из форматов, использующих эту раскладку, а не как её первооткрыватель. Специфичный для GoldenFloat вклад — это не сама раскладка

GF16, а вывод разбиения $e : f$ по всей лестнице от GF4 до GF256 из единого замкнутого правила $e = \text{round}((N - 1)/\varphi^2)$, для которого ступень $N = 16$ даёт $e = \text{round}(15/\varphi^2) = 6$, совпадая с раскладкой E6M9 DFloat как частный случай, плюс целочисленный тракт аккумулятора, точный по Люка (раздел 4). Совпадение лестничного правила на ступени $N = 16$ с независимо мотивированным E6M9 является свидетельством разумности правила на этой ступени, а не заявлением о приоритете или превосходстве.

ОСР-MX. Rouhani и др. (2023) определили форматы микромасштабирования Open Compute Project — отраслевое стандартное семейство блочно-масштабируемых 4-, 6- и 8-битных типов. Это развёрнутая эталонная точка для семейств малой разрядности.

LNS. Логарифмические системы счисления — логарифмический предшественник для систем не по основанию 2, с недавними экземплярами для машинного обучения в LNS-Madam (2021) и Alam, Garland и Gregg (2021). LNS даёт точное умножение ценой приближённого сложения; оно не нацелено на целочисленно-точное тождество аккумулятора в смысле раздела 4.

Арифметика Цекендорфа. Bergman (1957) ввёл систему счисления с φ в качестве иррационального основания — исторический предшественник числовых систем на основе φ . Ahlback, Usatine и Pippenger (2012) дают эффективные алгоритмы для арифметики в базисе Фибоначчи/ φ — алгоритмическую предшествующую работу для идеи целочисленного накопления раздела 4. Более недавнее предложение с учётом φ — квантование Fibbinary (Belghazi и др., 2025, arXiv:2511.01921), которое представляет веса в базисе Фибоначчи/ φ и сообщает о серьёзной деградации точности без восстановления через обучение с учётом квантования. Fibbinary работает на уровне поэлементного кодирования веса, а не как полное семейство форматов с плавающей точкой, поэтому оно дополняет GoldenFloat, а не является прямой альтернативой.

Аккумулятор Кулиша. Широкий аккумулятор с фиксированной точкой Кулиша (Kulisch, 2013) и его воплощение в виде quire posit (стандарт Posit 2022) — канонический эталон для точного накопления скалярного произведения. Точный по Люка тракт GoldenFloat не является аккумулятором Кулиша: он остаётся внутри базиса φ и использует тождество $\varphi^{2n} + \varphi^{-2n} = L_{2n}$ для сохранения частичных сумм в беззнаковом целочисленном хранилище, тогда как Кулиш хранит явный широкий регистр с фиксированной точкой и преобразует в него и из него.

Для более широкого контекста обзор числовых систем DNN (Alsuhli и др., 2023), определения форматов FP8 (Micikevicius и др., 2022), Huawei HiFloat8 (Luo и др., 2024), числовые форматы в климатических моделях (Kloewer и др., 2020), анализ масштабных законов точности (Kumar и др., 2024) и монография Gustafson The End of Error (2015) помещают эти семейства в более широкий ландшафт. На масштабе менее 1 млрд параметров BitNet b1.58 (Ma и др., 2024) и последующая работа 2B4T (Ma и др., 2025, arXiv:2504.12285) являются текущими чемпионами по бит-на-байт и естественным базисом ниже 1 млрд, относительно которого любое заявление о рецепте обучения GoldenFloat должно в конечном счёте измеряться.

Честное резюме. Нам не известно о другом опубликованном семействе форматов с плавающей точкой, которое выводит своё разбиение $e : f$ из единого замкнутого правила по лестнице от GF4 до GF256 с целочисленным тождеством, точным по Люка. Posit, takum, ОСР-MX, LNS и черновик многоразрядного формата с плавающей точкой IEEE P3109 — близкие предшественники и союзники, охватывающие ряд разрядностей собственными единичными параметризованными правилами; takum несёт ближайшее прошедшее рецензирование

многоразрядное аппаратное заявление, а IEEE P3109 несёт ближайшее усилие отраслевой стандартизации на малых разрядностях. Специфичный для GoldenFloat элемент — это комбинация статического разбиения $e : f$ из одного замкнутого правила и целочисленного тракта аккумулятора, точного по Люка, который зависит только от выбора основания φ , а не от разрядности.

7 Предрегистрация открытых гипотез

Дата и область авторства. Эта предрегистрация была составлена 2026-05-31 до того, как появились какие-либо кремниевые измерения при согласованном бюджете. Она заморожена на git-теге v1.9-prereg в gHashTag/goldenfloat-preprint (см. запись CHANGELOG для v1.9 для SHA-1 коммита) и не будет изменена после начала сбора данных при согласованном бюджете.

Четыре зарегистрированные гипотезы и их условия остановки таковы.

H_a (Широта-как-ров выдерживает контроль при согласованном бюджете). Нулевая: дельта BPB на бит на площадь кристалла между портфелем GoldenFloat и fp16-нативным базисом равного транзисторного бюджета равна нулю или отрицательна для GoldenFloat. Условие остановки: дельта BPB при согласованном бюджете падает ниже $\delta_{\text{thresh}} = 0.005$ бит-на-байт на согласованном корпусе оценки; объявляется фальсификация, и тезис широты-как-рова в разделе 5.4 отзывается.

H_b (Лестничное правило единственно внутри пространства поиска). Нулевая: по меньшей мере одно альтернативное правило отбора, построенное без обращения к лестнице GoldenFloat, воспроизводит те же 12 канонических разрядностей. Условие остановки: интервал поиска для сертификата единственности сужается до синглтонного множества; любое нетривиальное конкурирующее правило, опубликованное до v1.5, вносится в реестр фальсификации.

H_c (Смещение GF256 допускает вывод в замкнутой форме). Нулевая: не существует выражения в замкнутой форме, воспроизводящего константу смещения GF256 с полной двойной точностью IEEE-754 в аудируемой символической форме. Условие остановки: точная формула смещения помещена в аудируемой форме (вывод CAS + независимая выборочная проверка) в дополнительных материалах; формула должна пройти верификацию SymPy с точностью 500 знаков.

H_d (Кремний с исправленным RTL GF16 достигает паритета измерений). Нулевая: исправленный кристалл GF16 и fp16-нативный эталонный кристалл, согласованные по технологическому узлу и бюджету площади, различаются более чем на $X = 2\%$ по погрешности накопления туда-обратно (round-trip) на согласованных тестовых векторах. Условие остановки: парные измерения кристаллов из независимого испытательного дома попадают в пределы X процентов; пока не существует вернувшегося кристалла, эта гипотеза остаётся открытой, и нигде в препринте не делается кремниевого заявления.

8 Открытые вопросы и реестр фальсификации FL-002

Мы фиксируем открытые вопросы как реестр фальсификации, указывая для каждого статус, заявление и эксперимент, который его фальсифицировал бы. Отрицательные результаты излагаются первыми и прямо.

(а) Широта-как-ров (открытая гипотеза). Гипотеза состоит в том, что преимущество лестницы GF — это широта и согласованность инструментальной цепочки (одно замкнутое правило по всей лестнице), а не точность на отдельной ступени. Это не доказано абляцией, и единственные доступные программные данные BPB искажены субстратом (раздел 5.4). Фальсификация: (i) справедливое спаренное измерение — GF16-нативный кремний против fp16-нативного кремния при согласованном технологическом узле и площади, или FPGA-оснастка с эквивалентными бюджетами LUT/DSP для обоих форматов — в котором лестница posit, takum, MX или fp16 соответствует лестнице GF при согласованных бюджетах бит; (ii) F2, прямое сопоставление бит-на-байт конвейера смешанной точности φ -лестницы против гетерогенного зоопарка при согласованных бюджетах бит, с учётом потерь при межформатных преобразованиях; (iii) F3, контроль лестницы posit при тех же бюджетах бит. Любой отрицательный исход по (i)–(iii) должен быть зафиксирован прямо и первым.

(b) Неединственность лестничного правила (риск). Как показано в разделе 2.2, 83 значения отношений воспроизводят девять проверенных разрядностей; совпадающий интервал — $[0.3786, 0.3822]$, и $1/\varphi^2$ внутри него не выделяется. Фальсификация (или сужение): расширить множество проверенных разрядностей (например, добавить GF128 и GF512) и сообщить, сужается ли совпадающий интервал до синглтона или остаётся множественным.

(с) Хранимое смещение GF256 (открыто). Хранимое смещение приблизительно 2^{71} документировано для GF256 против ожидания в стиле IEEE $2^{96} - 1$ для 97-битного поля порядка. Это расхождение не объяснено в аудируемой форме. Фальсификация (или разрешение): опубликовать точную формулу смещения и её вывод.

(d) Кремний за пределами GF16 (открыто). Как документирует раздел 5.2, все десять разрядностей GF4 – GF256 несут синтезируемый RTL и были включены в отправку на shuttle TTSKY26b в Tiny Tapeout, но ни один физический кристалл не вернулся ни для одной ступени. Фальсификация (или разрешение) — возврат кремния и функциональная проверка на отдельной ступени против оракула соответствия формата раздела 5.3.

(е) Портфель умножителей в виде, отправленном в TTSKY26b (отозван). Как документируют раздел 5.5 и приложение F, RTL умножителя, отправленный на shuttle TTSKY26b 2026-05-17, несёт единственную ошибку формулы генератора (регистр произведения на два бита уже), реплицированную по разрядностям, помеченным крестом ($\dagger$). Поэтому изготовленные кристаллы gamma и phi будут нести дефектный портфель умножителей. Исправленный универсальный генератор является базовой версией для регенерации; статус отозван для отправки TTSKY26b, открыт в ожидании возврата кремния для следующего shuttle.

(f) FPGA-эксперимент на согласованном субстрате H_4 (запланирован). Кодеки GF16 и posit16, синтезированные на одном и том же Xilinx Artix-7 (XC7A100T, QMTECH Wukong V1) с согласованными бюджетами LUT/DSP, будут оценены по числу LUT, числу триггеров, $F_{\max}$ и динамической мощности на операцию умножения с накоплением. Предрегистрированные пороги (приложение D): $R_{\text{LUT}} < 1.15$ и $R_{F_{\max}} > 0.85$. Провал по любому порогу понижает широту-как-ров с открытой гипотезы до риска по оси кремния. Любой отрицательный результат сообщается первым и заметно.

9 Заключение

Лестничная арифметика воспроизводит девять реализованных ширин поля порядка 9 из 9 и согласованно распространяется на восемь дальнейших выведенных из правила ступеней — GF6, GF10, GF14, GF48, GF96, GF128, GF512, GF1024 (раздел 2); тождество Люка

$\varphi^{2n} + \varphi^{-2n} = L_{2n}$ выполняется точно для $n = 1, \dots, 256$, с якорем $\varphi^2 + \varphi^{-2} = 3 = L_2$, приписываемым Lucas (1878) и Бине (раздел 4); и единственный кодек GF16 проходит тестовое окружение 35 из 35 на частоте 323 МГц на Artix-7 (раздел 5.2). Широта-как-ров — заявление о том, что одно правило по всей лестнице даёт преимущество согласованности, а не преимущество точности на отдельной ступени, — остаётся открытой гипотезой (раздел 8 (а)). Что её фальсифицировало бы: лестница posit, takum или MX, соответствующая φ -лестнице при согласованных бюджетах бит. FPGA-эксперимент на согласованном субстрате (приложение D, предрегистрированный) обеспечит первое контролируемое аппаратное сравнение. Два дальнейших пункта открыты (смещение GF256 и кремний за пределами GF16) и один является риском (неединственность лестничного правила, с 83 совпадающими отношениями). Мы представляем эту заметку, чтобы установить цитируемую, фальсифицируемую запись проверенной арифметики и открытых вопросов.

А Верификационный скрипт F1 и расширенная таблица

Приведённый ниже скрипт проверяет $\varphi^{2n} + \varphi^{-2n} = L_{2n}$ символически (SymPy, точно в $\mathbb{Q}[\sqrt{5}]$) и численно (mpmath, 500 знаков) для $n = 1, \dots, 256$.

```
#!/usr/bin/env python3
"""F1: verify phi^(2n) + phi^(-2n) = L_{2n} (integer Lucas number)
for n = 1..256, both symbolically (sympy, exact) and numerically
(mpmath, 500 dps). Anchor: Ahlbach/Usatine/Pippenger (2012),
arXiv:1207.4497. The identity is a classical Binet corollary
(Lucas, 1878), not original to this work."""
from mpmath import mp, sqrt, power, nstr, mpf
from sympy import sqrt as ssqrt, simplify, Integer

def lucas_recurrence(max_index):
    L = [2, 1]
    for _ in range(2, max_index + 1):
        L.append(L[-1] + L[-2])
    return L[:max_index + 1]

def main():
    mp.dps = 500
    phi = (1 + sqrt(5)) / 2
    n_max = 256 # gives 2n = 2..512
    L = lucas_recurrence(2 * n_max)
    max_diff = mpf(0)
    all_pass = True
    for n in range(1, n_max + 1):
        m = 2 * n
        val = power(phi, m) + power(phi, -m)
        diff = abs(val - L[m])
        max_diff = max(max_diff, diff)
        all_pass = all_pass and (diff < mpf("1e-150"))
    # exact symbolic check in Q[sqrt(5)]
    phi_sym = (Integer(1) + ssqrt(5)) / 2
    exact_pass = all(
        simplify(phi_sym**(2*n) + phi_sym**(-2*n) - Integer(L[2*n])) == 0
        for n in range(1, n_max + 1))
    print("Anchor phi^2 + phi^-2 =",
          nstr(power(phi, 2) + power(phi, -2), 20), "= L_2 =", L[2])
    print("max |phi^(2n)+phi^(-2n) - L_2n| =", nstr(max_diff, 5))
    print("all numerical rows within 1e-150 ?", all_pass)
    print("exact symbolic identity holds for n=1..256 ?", exact_pass)
```

```

return 0 if all_pass else 1

if __name__ == "__main__":
    raise SystemExit(main())

```

В Индекс форматов GF (от GF4 до GF256)

Таблица 5 даёт для каждой реализованной разрядности: полное число бит N , биты порядка e , биты мантииссы f , отношение $e/(N-1)$ (цель $1/\varphi^2 = 0.38197$), phi-расстояние $|E/M - 1/\varphi|$, где $E/M = e/f$ и $1/\varphi = 0.61803$, и реализована ли разрядность.

С Таблица поправки на множественность

Таблица 6 сообщает для репрезентативных правил-кандидатов, сколько из девяти реализованных разрядностей ($N \in \{4, 8, 12, 16, 20, 24, 32, 64, 256\}$, с целевыми значениями $e \in \{1, 3, 4, 6, 7, 9, 12, 24, 97\}$) каждое воспроизводит. Исчерпывающий рациональный поиск по p/q с $p \in \{1, \dots, 99\}$, $q \in \{100, \dots, 499\}$ нашёл 83 различных значения отношений, совпадающих со всеми девятью; совпадающий интервал — $[0.3786, 0.3822]$. Правило GF round($(N-1)/\varphi^2$) — один член этого множества и не выделяется одним лишь воспроизведением (раздел 2.2).

D FPGA-эксперимент на согласованном субстрате (предрегистрированный)

Гипотеза H_4 . Кодеки GF16 и posit16, синтезированные на одном и том же Xilinx Artix-7 (XC7A100T-FGG676, QMTech Wukong V1) с согласованными бюджетами LUT/DSP, достигают сопоставимых числа LUT, числа триггеров, максимальной рабочей частоты ($F_{\max}$) и динамической мощности на операцию умножения с накоплением.

Предрегистрированные пороги. Пусть $R_{\text{LUT}} = \text{LUT}_{\text{GF16}}/\text{LUT}_{\text{posit16}}$ и $R_{F_{\max}} = F_{\max, \text{GF16}}/F_{\max, \text{posit16}}$. Эксперимент фальсифицирует H_4 тогда и только тогда, когда

$$R_{\text{LUT}} > 1.15 \quad \text{ИЛИ} \quad R_{F_{\max}} < 0.85.$$

Правило решения. Если условие фальсификации выполнено, широта-как-ров понижается с открытой гипотезы до риска по оси кремния в пункте (а) FL-002. Отрицательный результат сообщается первым, в аннотации или резюме отчёта о результатах, без задержки на опровержение. Если оба отношения в пределах границ, H_4 сохраняется, и статус широты-как-рва остаётся открытой гипотезой в ожидании измерения IGLA RACE на втором субстрате. Если GF16 превосходит posit16 по площади или частоте, это наблюдение фиксируется, но не повышает широту-как-ров до проверенной. Один кодек на одном субстрате — это не ров.

Материалы. RTL GF16: исправленные gf16_mul.v и gf16_add.v из gHashTag/tt-trinity-gamma PR #110. RTL posit16: PERCIVAL (Mallasen и др., 2022, arXiv:2111.15286). Программирование: Rust-драйвер cli/dlc10 (gHashTag/t27). Инструментальная цепочка: OpenXC7 (yosys + nextpnr-xilinx) или Vivado 2023.2. Шлюз корректности: исчерпывающая прогонка TestFloat-3, требуется 0 ошибок в 1 млн случайных выборок до принятия результатов по площади/таймингу.

Обязательство. Любой отрицательный результат — то есть любой исход, где GF16 требует более чем на 15% дополнительной площади или опускается ниже 85% от $F_{\max}$ posit16, — будет сообщён прямо и заметно. Никакой постфактумной корректировки порога, никакого избирательного опущения.

Дата предрегистрации: 2026-05-31. Полная спецификация SHA-256: см. git-тег v1.9-prereg в gHashTag/goldenfloat-preprint; SHA-256 рукописи на этом теге является авторитетным хешем предрегистрации.

Е Протокол независимого репликатора

Это приложение даёт точные шаги воспроизведения для каждого проверяемого заявления в препринте. Каждый шаг перечисляет команду, ожидаемый результат и заполнитель для дайджеста SHA-256.

Шаг R1: Проверка предложения 1 (тождество Люка). `python f1_lucas_proof_extended.py`. Ожидаемый результат: код выхода 0; вывод терминала включает строку MAX RESIDUAL < 1e-150 at 500 digits и строку F1 VERIFIED.

Шаг R2: Проверка лестничного правила (12/12 разрядностей). `python ladder_extended.py`. Ожидаемый результат: код выхода 0; вывод терминала включает LADDER 12/12 WIDTHS REPRODUCED и нет токенов FAIL.

```
git clone https://github.com/gHashTag/t27
cd t27/fpga/vsa/
dlc10 idcode
```

Ожидаемый idcode: 0x13631093.

```
dlc10 sram gf16_heartbeat_top.bit
```

Ожидаемый результат: консоль UART печатает GF16 HEARTBEAT ОК в течение 5 секунд после загрузки битстрима.

```
git clone --branch 110 \
    https://github.com/gHashTag/tt-trinity-gamma tt-trinity-gamma-110
cd tt-trinity-gamma-110
bash verification/run_gf_audit.sh
```

Ожидаемый результат: код выхода 0; вывод терминала содержит точную строку GF AUDIT ALL PASS на последней непустой строке.

F Errata о корректности RTL: расширенные подробности

Это приложение даёт полную помодульную разбивку, вывод корневой причины и объём исправления дефекта портфеля умножителей от 2026-05-31, кратко изложенного в разделе 5.5. Дефект был найден дифференциальной прогонкой на уровне RTL относительно корректно округлённого эталона с плавающей точкой в репозиториях чипов TRI-NET (gHashTag/tt-trinity-gamma и gHashTag/tt-trinity-phi), применённой к сгенерированному портфелю умножителей, отправленному на shuttle TTSKY26b Tiny Tapeout 2026-05-17.

Затронутые модули. `gf8_mul` не проходит на $\approx 95\%$ исчерпывающей прогонки из 26,360 точек входа; `gf12_mul` не проходит на $\approx 99\%$ прогонки из 109,576 точек; `gf20_mul`, `gf24_mul` и `gf32_mul` каждый не проходят на 100% направленных входов (например, 1.0×1.0 читается как 0.5, порядок смещён на единицу); `gf16_mul` корректен только после однострочного расширения регистра округлённой мантиссы с 9 до 10 бит, и вариант в виде, отправленном, несёт ошибку переполнения округления; `gf64_mul`, `gf128_mul` и `gf256_mul` разделяют то же семейство генератора и предположительно затронуты по той же причине, хотя они ещё не были исчерпывающе прогнаны. `gf4_mul` вырожден ($e = 1$ не оставляет нормальных порядков) и не затронут. Сумматоры `gf8_add` и `gf12_add` имели отдельный дефект нормализации узкого формата (например, $0.25 + 0.25$ читается как 0); более широкие сумматоры `gf16_add` и `gf32_add` были уже корректны.

Корневая причина. Произведение двух $(M + 1)$ -битных мантисс (с неявной ведущей единицей) имеет ширину $2M + 2$ бита и попадает в $[2^{2M}, 2^{2M+2})$. Отправленный RTL объявил регистр произведения на два бита уже и выполнял нормализацию на битах, сдвинутых вниз на два. Для самого базового входа $1.0 \times 1.0 = 2^{2M}$ ведущий бит усекается, порядок декрементируется, и результат читается как 0.5. Это единственная ошибка формулы генератора (позиция старшего бита должна быть $2M + 1$), реплицированная по всему портфелю.

Почему покрытие формальными методами это не поймало. База доказательств Coq в репозиториях чипов TRI-NET устанавливает математику уровня формата GoldenFloat (тождество $\varphi^2 + \varphi^{-2} = 3$, тождества кодирования, согласованность лестничного правила); она не устанавливает ширины бит и округление реализации на Verilog. Дефект целиком живёт в реализации RTL, в коде вне доказанного ядра. Это усиливает, а не ослабляет позицию верифицируемости настоящей заметки: машинно проверенная математика уровня формата необходима, но недостаточна; дифференциальная прогонка на уровне RTL относительно корректно округлённого эталона должна быть частью конвейера аудита.

Исправление и текущий статус. Исправленный универсальный генератор умножителя выпускает правильный шаблон для любого (E, M, BIAS) : ширина произведения $[2M + 1 : 0]$, нормализация на $[2M + 1]$ или $[2M]$, извлечение мантиссы $[2M : M + 1]$ или $[2M - 1 : M]$, округление полупозиции вверх с переносом. Исправленный портфель чист: `gf8` исчерпывающая прогонка 0 отказов из 26,360; `gf12` 0 из 109,576; `gf16` 0 из 262,144; `gf20` 0 из 100,000; `gf24` 0 из 100,000; `gf32` 0 из 200,000; `gf64`, `gf128`, `gf256` проходят направленные точные тесты (1.0×1.0 , 1.5×1.5 , ...), хотя полные прогонки остаются будущей работой. Исправленный RTL опубликован как запросы на слияние `gHashTag/tt-trinity-gamma#110` (54 файла, +2452/-1032) и `gHashTag/t27#1024` (16 файлов, +558/-10), а методологическая заметка опубликована как `gHashTag/trinity-clara#20`. CI-шлюз (`run_gf_audit.sh`, встроенный в `.github/workflows/gf-audit.yml`) теперь повторно прогоняет дифференциальную прогонку при каждом коммите.

Объём исправления. Кремний TTSKY26b уже отправлен, и изготовление не может быть отозвано; поэтому изготовленные кристаллы `gamma` и `phi` будут нести дефектный RTL умножителя. Канонический якорь POST 0x47C0 и тракты тождества формата должны быть повторно проверены на вернувшемся кремнии, но ни одно заявление о точности на отдельной ступени в данной заметке в любом случае не зависело от портфеля умножителей в отправленном виде (раздел 5.2 документировал все ступени, кроме GF16, как только RTL). Исправленный портфель является базовой версией для регенерации для следующего shuttle. Дефект не отменяет лестничную арифметику раздела 2, тождество Люка раздела 4 или учёт множественности приложения C, которые являются абстрактными спецификациями, не зависящими от какой-либо конкретной реализации RTL.

Благодарности

Laslo Hunhold (Кёльнский университет) предоставил подробную обратную связь по версии v1.7 данной рукописи, которая сформировала настоящую ревизию: более короткую двухчастную аннотацию; расширенное введение, обрамляющее компромисс динамического диапазона и мотивирующее выведенное из φ разбиение до формулировки правила; абзац мотивации (потеря значимости при большом накоплении и сравнение с quire/Кулиш), помещённый перед предложением 1; использование предложения вместо эпистемических меток в основном тексте; определение лестницы GF при $N \geq 4$ с $N \in \{2, 3\}$, отмеченными как вырожденные краевые случаи формулы; и единообразную терминологическую замену мантиссы (fraction) на мантиссу повсюду. Ни одна из оставшихся переоценок, ошибок или открытых гипотез не должна приписываться Hunhold; они являются ответственностью автора.

Список литературы

- [1] G. Bergman, “A number system with an irrational base,” *Mathematics Magazine*, vol. 31, pp. 98–110, 1957.
- [2] I. Daubechies, C. S. Gunturk, Y. Wang, and O. Yilmaz, “The Golden Ratio Encoder,” *IEEE Transactions on Information Theory*, vol. 56, no. 10, pp. 5097–5110, 2010. DOI 10.1109/TIT.2010.2059750. arXiv:0809.1257.
- [3] R. Ward, “On robustness properties of beta encoders and golden ratio encoders,” 2008. arXiv:0806.1083.
- [4] J. L. Gustafson and I. Yonemoto, “Beating floating point at its own game: posit arithmetic,” *Supercomputing Frontiers and Innovations*, vol. 4, no. 2, 2017. DOI 10.14529/JSFI170206.
- [5] Posit Working Group, “Standard for Posit Arithmetic,” 2022. https://posithub.org/docs/posit_standard-2.pdf.
- [6] F. de Dinechin, L. Forget, J.-M. Muller, and Y. Uguen, “Posits: the good, the bad and the ugly,” 2019. hal-03195756v3.
- [7] L. Hunhold, “Takum arithmetic,” 2024. arXiv:2404.18603.
- [8] L. Hunhold, “Integer representations of takums,” 2024. arXiv:2412.20273.
- [9] L. Hunhold, “A VHDL codec for takum arithmetic,” 2024. arXiv:2408.10594.
- [10] L. Hunhold and S. Quinlan, “Bfloat16, posit and takum number formats in linear-solver workloads,” 2024. arXiv:2412.20268.
- [11] L. Hunhold, “Hardware evaluation of takum arithmetic,” *ARITH 2025 (IEEE 32nd Symposium on Computer Arithmetic)*. DOI 10.1109/ARITH64983.2025.00019.
- [12] D. Mallasen et al., “PERCIVAL: open-source posit RISC-V core,” *IEEE Transactions on Emerging Topics in Computing*, 2022. DOI 10.1109/TETC.2022.3187199. arXiv:2111.15286.
- [13] D. Mallasen et al., “Big-PERCIVAL: exploring the native use of 64-bit posit arithmetic,” *IEEE Transactions on Computers*, 2024. DOI 10.1109/TC.2024.3377890. arXiv:2305.06946.
- [14] B. D. Rouhani et al., “Microscaling data formats for deep learning” (OCP-MX), 2023. arXiv:2310.10537.
- [15] J. Zhao et al., “LNS-Madam: low-precision training in logarithmic number system,” 2021. arXiv:2106.13914.
- [16] M. S. Alam, J. Garland, and D. Gregg, “Low-precision logarithmic number systems,” 2021. arXiv:2102.06681.

- [17] C. Ahlback, J. Usatine, and N. Pippenger, “Efficient algorithms for Zeckendorf arithmetic,” 2012. arXiv:1207.4497.
- [18] H. F. Alsuhli et al., “Number systems for deep neural network architectures: a survey,” 2023. arXiv:2307.05035.
- [19] P. Micikevicius et al., “FP8 formats for deep learning,” 2022. arXiv:2209.05433.
- [20] Y. Luo et al., “Ascend HiFloat8 format for deep learning,” 2024. arXiv:2409.16626.
- [21] A. Agrawal et al., “DLFloat: a 16-b floating point format designed for deep learning training and inference,” ARITH 2019 (IEEE 26th Symposium on Computer Arithmetic), pp. 92–95. DOI 10.1109/ARITH.2019.00023. (16-bit format with 6 exponent and 9 mantissa bits; the GF16 E6M9 layout.)
- [22] V. Popescu, A. Sentieys et al., “A 16-bit floating point format with 1/6/9 bit allocation for deep learning,” 2021. arXiv:2103.15940. (Independent motivation of the 1/6/9 split.)
- [23] M. Kloewer, P. D. Duben, and T. N. Palmer, “Number formats, error mitigation, and scope for 16-bit arithmetics in weather and climate modeling,” *Journal of Advances in Modeling Earth Systems*, vol. 12, 2020. DOI 10.1029/2020MS002246.
- [24] S. Ma et al., “The era of 1-bit LLMs: BitNet b1.58,” 2024. arXiv:2402.17764.
- [25] J. L. Gustafson, *The End of Error: Unum Computing*. CRC Press, 2015. ISBN 9781482239874.
- [26] T. Kumar et al., “Scaling laws for precision,” 2024. arXiv:2411.04330.
- [27] S. Ma et al., “BitNet b1.58 2B4T technical report,” 2025. arXiv:2504.12285.
- [28] M. I. Belghazi et al., “Fibbinary: quantization of neural networks using Fibonacci representations,” 2025. arXiv:2511.01921.
- [29] IEEE P3109 Working Group, “Standard for binary floating-point arithmetic for machine learning,” working draft v0.9.1, 2025. Reference implementation: graphcore-research/gfloat.
- [30] U. Kulisch, *Computer Arithmetic and Validity: Theory, Implementation, and Applications*, 2nd ed. De Gruyter, 2013. ISBN 9783110301731.

Таблица 3: Лестница GoldenFloat против takum (Hunhold 2024) — структурированное сравнение. Это не конкурентный рейтинг; превосходство по точности не заявляется.

Ось	Лестница GoldenFloat (GF)	Takum (Hunhold 2024)
Тип разбиения	Статическое: ширины полей порядка и мантиссы фиксированы для каждой ступени замкнутым правилом $e = \text{round}((N - 1)/\varphi^2)$; не зависят от значения.	Конусное (tapered): единая вогнутая проекция отображает полный целочисленный диапазон на интервал числовой прямой; ширина мантиссы варьируется поэлементно.
Единое замкнутое правило	Да — лестничное правило $N \mapsto (e, f)$ детерминировано; нет поэлементного ветвления на уровне формата.	Да — проекция takum — это единое аналитическое отображение, применяемое единообразно к каждому n -битному целому.
Базовая константа	Золотое сечение $\varphi = (1 + \sqrt{5})/2$; ширины полей порядка растут как дроби последовательных чисел Фибоначчи от полной рядности слова.	Основание натурального логарифма e : takum — логарифмический формат конусной точности, чьё кодированное значение является функцией $\ln(x)$, с полями знак + режим + характеристика + мантисса (Hunhold 2024).
Статус аппаратной части	RTL-портфель существует (исправлен после обнаружения дефекта умножителя в дифференциальной прогонке; см. раздел 5.5); синтез FPGA сообщён внут-	RTL- и FPGA-результаты прошли рецензирование на ARITH 2025 (Hunhold 2024/2025); кремний не сообщён.

Таблица 4: Расширенный вывод F1. $\varphi^{2n} + \varphi^{-2n}$ равно целому числу Люка L_{2n} с точностью 500 знаков. Избранные строки из полного прогона $n = 1, \dots, 256$; остатки абсолютные.

n	$2n$	L_{2n}	остаток
1	2	3	0
2	4	7	1.39×10^{-501}
4	8	47	1.66×10^{-501}
8	16	2207	4.53×10^{-501}
16	32	4870847	8.4×10^{-501}
32	64	23725150497407	1.99×10^{-500}
64	128	5.629×10^{26}	3.75×10^{-500}
128	256	3.168×10^{53}	7.67×10^{-500}
192	384	1.783×10^{80}	1.15×10^{-499}
256	512	1.004×10^{107}	1.55×10^{-499}

Таблица 5: Индекс форматов GF для девяти реализованных разрядностей.

N	e	f	$e/(N-1)$	$ E/M - 1/\varphi $	реализ.?
4	1	2	0.33333	0.11803	Да
8	3	4	0.42857	0.13197	Да
12	4	7	0.36364	0.04661	Да
16	6	9	0.40000	0.04863	Да
20	7	12	0.36842	0.03470	Да
24	9	14	0.39130	0.02482	Да
32	12	19	0.38710	0.01354	Да
64	24	39	0.38095	0.00265	Да
256	97	158	0.38039	0.00411	Да

Таблица 6: Правила-кандидаты и их счёт воспроизведения девяти разрядностей.

Правило	Совпадения	Примечание
$\text{round}((N-1)/\varphi^2)$	9/9	правило GF; отношение ≈ 0.38197
$\lfloor N/\varphi^2 \rfloor$	9/9	столь же просто; то же отношение
$\text{round}((N-1) \cdot 0.382)$	9/9	округлённая константа; без φ
$\text{round}((N-1) \cdot 3/7.85)$	9/9	произвольная рациональная; без φ
$\text{round}((N-1) \cdot 3/8)$	8/9	не проходит GF256 (96 против 97)
$\text{round}((N-1) \cdot 5/13)$	8/9	не проходит GF256
$\lfloor N \cdot 3/8 \rfloor$	8/9	не проходит GF256
$\text{round}((N-1)/2.6)$	8/9	не проходит GF256
$\text{round}((N-1)/e)$	5/9	не проходит $N=24,32,64,256$
$\lfloor (N-1)/\varphi^2 \rfloor$	5/9	floor против round имеет значение
$\text{round}((N-1)/\pi)$	2/9	плохое соответствие
$\text{round}((N-1)/\varphi)$	0/9	неверная константа

**GoldenFloat: a family of static-split floating-point formats
derived from the golden ratio φ , from GF4 to GF256,
with an exact integer Lucas identity**

D. V. Vasilev

Independent researcher, Ko Samui, Thailand

e-mail: admin@t27.ai ORCID: 0009-0008-4294-6159

Abstract. We present a hardware-oriented description of GoldenFloat (GF) — a family of static-split floating-point formats generated by a single closed-form rule — together with three concrete artefacts: (i) an open multi-width RTL generator covering GF4–GF256 with a continuous-integration differential sweep against a correctly-rounded reference; (ii) a Lucas-exact integer accumulator path verified to 500 digits for $n = 1, \dots, 256$; and (iii) a GF16 FPGA codec passing a 35/35 testbench at 323 MHz on Artix-7 (Xilinx XC7A100T). For every full width $N \geq 4$ the exponent-field width is $e = \text{round}((N - 1)/\varphi^2)$ with fraction width $f = N - 1 - e$ and $\varphi = (1 + \sqrt{5})/2$. The rule reproduces the realised exponent widths of nine formats (9/9) and extends consistently to further widths. It is positioned alongside posit (Posit Standard 2022), takum (Hunhold 2024, 2025), OCP-MX (Rouhani et al. 2023) and the IEEE P3109 multi-width floating-point draft. The breadth/toolchain-coherence thesis is registered as an open conjecture with a pre-registered falsification path (ledger FL-002); we make no per-step accuracy or superiority claim. An RTL correctness erratum (2026-05-31) is reported: the fabricated TTSKY26b dies carry a defective multiplier portfolio, and the corrected generator is the baseline for regeneration.

Keywords: numeric formats; floating point; golden ratio; Lucas numbers; posit; takum; OCP-MX; arithmetic hardware.

References: see the bibliography list above; the English “References” list is produced by the publisher from the same entries.